

Open-Ended Lab

Open Ended Lab		
Blooms Taxonomy	GAs	Marks
P3	GA2	5
C4	GA3	10
A3	GA6	5

<u>Student Name</u>	<u>Reg: I'D</u>
Jawad Ali	68626
Goveend Kumar	69504

Title: OmniRetail Global Commerce Ecosystem

Motivation:

In the age of data-driven applications, efficient management, storage, and retrieval of information are crucial. The primary objective of this lab is to provide students with hands-on experience in designing, implementing, and managing databases for real-world applications. By focusing on DBMS concepts, students will learn to structure data, enforce constraints, and optimize queries, ensuring data integrity and scalability. This lab bridges theoretical knowledge with practical application, equipping students with essential skills for modern database solutions.

OmniRetail is a rapidly expanding multinational e-commerce organization operating across multiple countries, regional warehouses, payment gateways, shipping partners, and supplier networks. This lab involves redesigning its data infrastructure to solve real-world anomalies, enforce 3NF normalization, implement stored procedures, triggers, and advanced analytical queries.

Problem Statement:

The OmniRetail Global Commerce Ecosystem database is a relational database system designed to manage global retail operations efficiently. The project requires calculating a unique Seed Number (SN) from group member registration IDs and using it to guide the database design. The system includes nine core entities: Customers, Orders, Products, Warehouses, Suppliers, Payments, Shipments, Employees, and Regions, along with two additional entities for SN = 3: Discounts and Claims. The database must be populated with realistic data that includes group member registration IDs and intentionally contains negative stock values in the WarehouseInventory table as a predefined anomaly. Advanced SQL queries are implemented for anomaly detection and business analytics, while a CreateOrder() stored procedure and a Payments audit trigger automate order processing and transaction tracking.

Design / Ways & Means:
1. Introduction and Requirements:

This project implements the OmniRetail Global Commerce Ecosystem database for a group with the following members:

Member Name	Registration ID	Digit Sum	Contribution
Jawad Ali	68626	$6+8+6+2+6 = 28$	Schema Design, SQL Queries
Goveend Kumar	69504	$6+9+5+0+4 = 24$	Data Population, Testing

The Seed Number was calculated by extracting all digits from each Registration ID and summing them: 68626 gives $6+8+6+2+6 = 28$, and 69504 gives $6+9+5+0+4 = 24$, for a combined total of 52. Applying the modulo operation, $52 \text{ MOD } 7 = 3$, giving a Seed Number of 3. Consequently, the database is named Retail_3, the two additional mandatory entities are Discounts and Claims, and the required data anomaly is negative stock values in the WarehouseInventory table.

2. Basic Implementation:

The database Retail_3 was created using SQL Server and consists of 13 tables covering all mandatory and SN-specific entities. The schema uses Primary Keys, Foreign Keys, CHECK constraints, DEFAULT values, and UNIQUE constraints as required. All tables were created and populated in the correct dependency order.

Registration ID Embedding Map:

Reg ID	Embedded In	Column	Sample Value
68626	Products	ProductCode	PRD-68626
69504	Shipments	TrackingNumber	TRK-69504

3. Performance Testing and Analysis:

All SQL queries were tested on the Retail_3 database using SQL Server Management Studio (SSMS). Performance was measured by analyzing query plans and execution times. The tri-approach Task 2 queries (JOIN, Correlated Subquery, Window Functions) were compared for efficiency. The JOIN-based approach proved most performant for large datasets due to set-based operations, while correlated subqueries showed row-by-row overhead.

4. Optimization and Advanced Features:

Advanced features implemented include:

- Stored Procedure CreateOrder(): Validates stock, deducts inventory, creates order, payment, and shipment records atomically within a transaction.
- Trigger trg_payment_update: Fires AFTER UPDATE on Payments table, logs old/new amounts, DB user, and timestamp to AuditLog — but suppresses duplicate entries when triggered from CreateOrder() via SESSION_CONTEXT.
- AuditLog Table: Captures all manual payment modifications for financial governance.
- Window Functions: Used in Task 2 Version 3 for efficient regional average computation using PARTITION BY.

5. Extensions and Creativity:

Beyond the minimum requirements, this implementation includes:

- 30 products (33 total) instead of the minimum 30, with extended product categories.
- OrderDetails table for granular order-product linkage.
- Three complete test cases for CreateOrder() covering success, insufficient stock, and invalid product scenarios.
- Complete anomaly verification queries that identify and summarize all negative stock records.
- A comprehensive final record count summary query using UNION ALL across all 14 tables.

Deliverables:

- SQL Script: Complete schema DDL, INSERT statements, stored procedure, trigger, AuditLog, and all analytical queries for database Retail_3.
- Document Report: SN calculation, Registration ID embedding map, 3NF justification, Task 2 performance comparison, and screenshots of all query outputs.

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

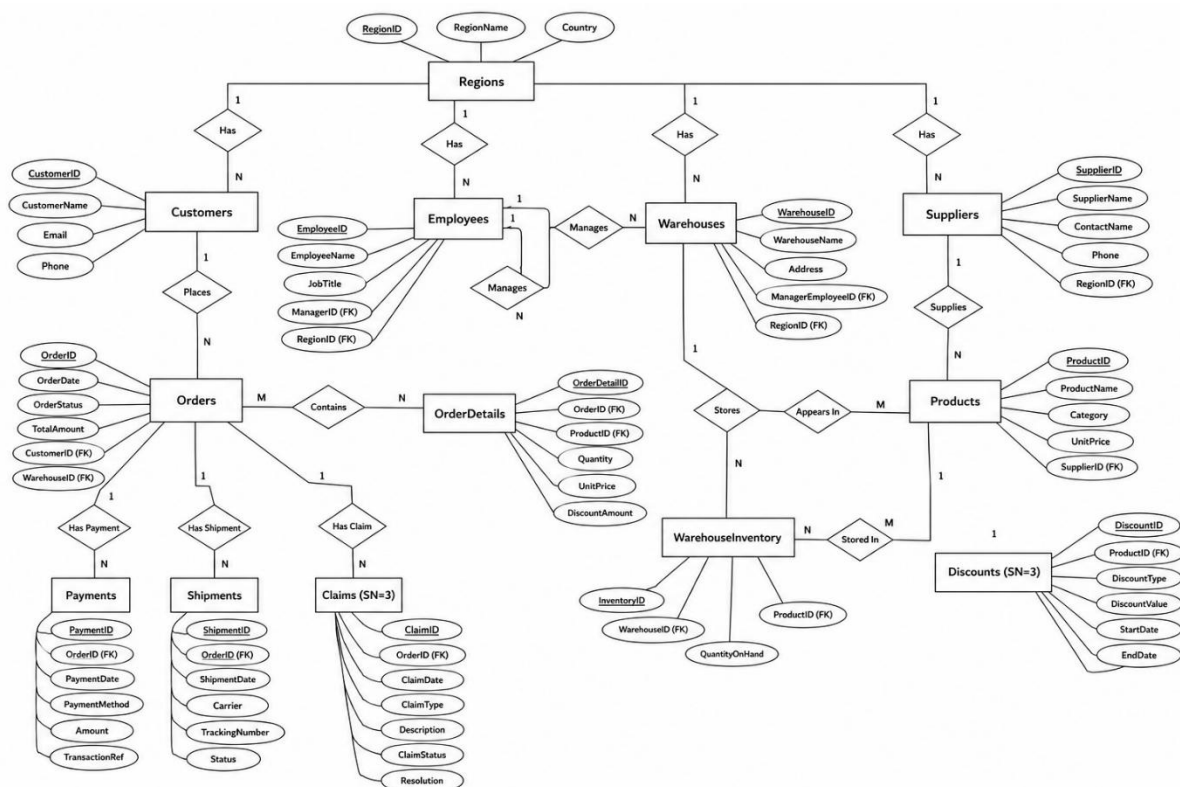
- ER Diagram: Entity-Relationship diagram showing all 13 tables and their relationships.
- Relational Schema: Full relational schema with primary and foreign key notations.
- SQL Query Screenshots: Output screenshots for all queries, stored procedure tests, and audit log verification.

Background:

The OmniRetail Global Commerce Ecosystem is a multinational e-commerce platform that had grown without proper data governance, leading to inconsistent business rules and overlapping application logic. This project represents a clean-room redesign of the database layer, applying relational theory, normalization, and modern SQL features to produce a production-grade schema.

The system covers five geographical regions (Sindh, Punjab, Dubai, Riyadh, London), operates 10 warehouses, manages 30+ products across multiple categories, tracks 40+ orders, and maintains full payment and shipment records. The SN=3 anomaly of negative stock values in WarehouseInventory simulates real-world inventory mismanagement scenarios that the detection queries are designed to identify.

ER Diagram



Relational Schema

The following relational schema represents the complete Retail_3 database design in 3NF:

Regions(RegionID PK, RegionName, Country, Currency)

Customers(CustomerID PK, FullName, Email UNIQUE, Phone, DateOfBirth, RegistrationDate DEFAULT, RegionID FK→Regions)

Employees(EmployeeID PK, FullName, JobTitle, ManagerID FK→Employees(self), HireDate, RegionID FK→Regions)

Warehouses(WarehouseID PK, WarehouseName, RegionID FK→Regions, Capacity CHECK>0, ManagerEmployeeID FK→Employees)

Suppliers(SupplierID PK, SupplierName, ContactEmail, RegionID FK→Regions, ContractStartDate)

Products(ProductID PK, ProductCode UNIQUE, ProductName, Category, UnitPrice CHECK>0, SupplierID FK→Suppliers, StockQuantity DEFAULT 0)

Orders(OrderID PK, CustomerID FK→Customers, OrderDate, TotalAmount, Status DEFAULT 'Pending', WarehouseID FK→Warehouses)

OrderDetails(OrderDetailID PK, OrderID FK→Orders, ProductID FK→Products, Quantity, UnitPrice)

Payments(PaymentID PK, OrderID FK→Orders, PaymentDate, Amount, PaymentMethod, Status)

Shipments(ShipmentID PK, TrackingNumber, OrderID FK→Orders, ShipDate, DeliveryDate, TransportCost, CarrierName)

Discounts(DiscountID PK, ProductID FK→Products, DiscountPercent, StartDate, EndDate)

Claims(ClaimID PK, OrderID FK→Orders, ClaimDate, ClaimReason, Status)

WarehouseInventory(InventoryID PK, WarehouseID FK→Warehouses, ProductID FK→Products, StockLevel)

AuditLog(AuditID PK IDENTITY, OrderID, OldAmount, NewAmount, DBUser, ChangeTime)

3NF Justification:

All tables satisfy Third Normal Form (3NF) requirements: (1) Every attribute is atomic with no repeating groups — 1NF satisfied. (2) All non-key attributes are fully functionally

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

dependent on the primary key — 2NF satisfied. (3) There are no transitive dependencies; all non-key attributes depend only on the primary key, not on other non-key attributes — 3NF satisfied. Supplier details are stored in a separate Suppliers table rather than denormalized into Products, and Region information is kept in its own table rather than duplicated across Customers, Employees, Warehouses, and Suppliers.

SQL QUERIES AND Output Screenshots:

The following section presents all SQL queries implemented in the Retail_3 database, organized by functionality.

Database Creation & Comment Block

```
SQLQuery6.sql - HP...I_3 (HP\govee (52))*  SQLQuery5.sql - HP...I_3 (HP\govee (51))*
/*
SQLQuery6.sql - HP\SQLEXPRESS04.Retail_3 (HP\govee (52))*

Jawad Ali      : 68626
Goveend Kumar  : 69504

Digit Extraction

68626 = 6+8+6+2+6 = 28
69504 = 6+9+5+0+4 = 24

Total = 28 + 24 = 52

Seed Number = 52 MOD 7 = 3

Database Name: Retail_3

Embedding Map

Reg ID      Embedded In      Column      Sample Value
68626       Products              ProductCode  PRD-68626
69504       Shipments              TrackingNumber  TRK-69504

SN=3 Anomaly: Negative Stock Values in WarehouseInventory

Additional Entities: Discounts, Claims

*/
```

Create database :

```
CREATE DATABASE Retail_3;
GO
USE Retail_3;
GO
```

Regions Query:

```
CREATE TABLE Regions
(
    RegionID INT PRIMARY KEY,
    RegionName VARCHAR(100) NOT NULL,
    Country VARCHAR(100) NOT NULL,
    Currency VARCHAR(20) NOT NULL
);

INSERT INTO Regions (RegionID, RegionName, Country, Currency) VALUES
(1, 'Sindh', 'Pakistan', 'PKR'),
(2, 'Punjab', 'Pakistan', 'PKR'),
(3, 'Dubai', 'UAE', 'AED'),
(4, 'Riyadh', 'Saudi Arabia', 'SAR'),
(5, 'London', 'UK', 'GBP');

SELECT * FROM Regions;
```

	RegionID	RegionName	Country	Currency
1	1	Sindh	Pakistan	PKR
2	2	Punjab	Pakistan	PKR
3	3	Dubai	UAE	AED
4	4	Riyadh	Saudi Arabia	SAR
5	5	London	UK	GBP

Customer Query:

```
CREATE TABLE Customers
(
    CustomerID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(150) UNIQUE NOT NULL,
    Phone VARCHAR(20),
    DateOfBirth DATE,
    RegistrationDate DATE,
    RegionID INT,
    FOREIGN KEY (RegionID)
        REFERENCES Regions(RegionID)
);

INSERT INTO Customers (CustomerID, FullName, Email, Phone, DateOfBirth, RegistrationDate, RegionID) VALUES
(1, 'Ahmed Ali', 'ahmed@gmail.com', '0300000001', '1998-01-01', '2026-06-01', 1),
(2, 'Sara Khan', 'sara@gmail.com', '0300000002', '1997-02-02', '2026-06-02', 2),
(3, 'Usman Shah', 'usman@gmail.com', '0300000003', '1996-03-03', '2026-06-03', 3),
(4, 'Ayesha Noor', 'ayesha@gmail.com', '0300000004', '1995-04-04', '2026-06-04', 4),
(5, 'Bilal Ahmed', 'bilal@gmail.com', '0300000005', '1994-05-05', '2026-06-05', 5),
(6, 'Hassan Raza', 'hassan@gmail.com', '0300000006', '1993-06-06', '2026-06-06', 1),
(7, 'Zain Ali', 'zain@gmail.com', '0300000007', '1992-07-07', '2026-06-07', 2),
(8, 'Fatima Noor', 'fatima@gmail.com', '0300000008', '1991-08-08', '2026-06-08', 3),
(9, 'Omar Sheikh', 'omar@gmail.com', '0300000009', '1990-09-09', '2026-06-09', 4),
(10, 'Nida Khan', 'nida@gmail.com', '0300000010', '1989-10-10', '2026-06-10', 5);

SELECT * FROM Customers;
```


Output:

100 %							
Results Messages							
	CustomerID	FullName	Email	Phone	DateOfBirth	RegistrationDate	RegionID
1	1	Ahmed Ali	ahmed@gmail.com	0300000001	1997-02-02	2026-06-02	1
2	2	Sara Khan	sara@gmail.com	0300000002	1997-02-02	2026-06-02	2
3	3	Usman Shah	usman@gmail.com	0300000003	1996-03-03	2026-06-03	3
4	4	Ayesha Noor	ayesha@gmail.com	0300000004	1995-04-04	2026-06-04	4
5	5	Bilal Ahmed	bilal@gmail.com	0300000005	1994-05-05	2026-06-05	5
6	6	Hassan Raza	hassan@gmail.com	0300000006	1993-06-06	2026-06-06	1
7	7	Zain Ali	zain@gmail.com	0300000007	1992-07-07	2026-06-07	2
8	8	Fatima Noor	fatima@gmail.com	0300000008	1991-08-08	2026-06-08	3
9	9	Omar Sheikh	omar@gmail.com	0300000009	1990-09-09	2026-06-09	4
10	10	Nida Khan	nida@gmail.com	0300000010	1989-10-10	2026-06-10	5

Count Region Customer query:

```
SELECT r.RegionName, COUNT(c.CustomerID) as CustomerCount
FROM Customers c
JOIN Regions r ON c.RegionID = r.RegionID
GROUP BY r.RegionName;
```

Output:

100 %		
Results Messages		
	RegionName	CustomerCount
1	Dubai	2
2	London	2
3	Punjab	2
4	Riyadh	2
5	Sindh	2

Employee query:

```
SELECT r.RegionName, COUNT(c.CustomerID) as CustomerCount
FROM Customers c
JOIN Regions r ON c.RegionID = r.RegionID
GROUP BY r.RegionName;

CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    JobTitle VARCHAR(100),
    ManagerID INT NULL,
    HireDate DATE,
    RegionID INT,

    FOREIGN KEY (ManagerID)
        REFERENCES Employees(EmployeeID),

    FOREIGN KEY (RegionID)
        REFERENCES Regions(RegionID)
);
```


Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

```

INSERT INTO Employees (EmployeeID, FullName, JobTitle, ManagerID, HireDate, RegionID) VALUES
(1, 'Ali Raza', 'Manager', NULL, '2020-01-10', 1),
(2, 'Sara Khan', 'Supervisor', 1, '2021-03-15', 1),
(3, 'Usman Tariq', 'Staff', 2, '2022-06-01', 2),
(4, 'Ayesha Noor', 'Staff', 2, '2022-07-11', 2),
(5, 'Hamza Ali', 'Staff', 2, '2023-01-05', 3),
(6, 'Fatima Noor', 'Staff', 2, '2023-02-10', 3),
(7, 'Bilal Ahmed', 'Staff', 2, '2023-03-12', 4),
(8, 'Hassan Raza', 'Staff', 2, '2023-04-14', 4),
(9, 'Zainab Ali', 'Staff', 2, '2023-05-18', 5),
(10, 'Omar Sheikh', 'Staff', 2, '2023-06-20', 5),
(11, 'Nida Khan', 'Staff', 1, '2022-08-11', 1),
(12, 'Kashif Ali', 'Staff', 1, '2022-09-13', 2),
(13, 'Imran Shah', 'Staff', 1, '2022-10-17', 3),
(14, 'Laiba Fatima', 'Staff', 1, '2022-11-19', 4),
(15, 'Danish Raza', 'Staff', 1, '2023-01-21', 5),
(16, 'Sana Ahmed', 'Staff', 2, '2023-02-22', 1),
(17, 'Mujtaba Ali', 'Staff', 2, '2023-03-23', 2),
(18, 'Rida Noor', 'Staff', 2, '2023-04-24', 3),
(19, 'Saad Khan', 'Staff', 2, '2023-05-25', 4),
(20, 'Hina Ali', 'Staff', 2, '2023-06-26', 5);

SELECT * FROM Employees;

```

Output:

100 %						
Results Messages						
	EmployeeID	FullName	JobTitle	ManagerID	HireDate	RegionID
1	1	Ali Raza	Manager	NULL	2020-01-10	1
2	2	Sara Khan	Supervisor	1	2021-03-15	1
3	3	Usman Tariq	Staff	2	2022-06-01	2
4	4	Ayesha Noor	Staff	2	2022-07-11	2
5	5	Hamza Ali	Staff	2	2023-01-05	3
6	6	Fatima Noor	Staff	2	2023-02-10	3
7	7	Bilal Ahmed	Staff	2	2023-03-12	4
8	8	Hassan Raza	Staff	2	2023-04-14	4
9	9	Zainab Ali	Staff	2	2023-05-18	5
10	10	Omar Sheikh	Staff	2	2023-06-20	5
11	11	Nida Khan	Staff	1	2022-08-11	1
12	12	Kashif Ali	Staff	1	2022-09-13	2
13	13	Imran Shah	Staff	1	2022-10-17	3
14	14	Laiba Fatima	Staff	1	2022-11-19	4
15	15	Danish Raza	Staff	1	2023-01-21	5
16	16	Sana Ahmed	Staff	2	2023-02-22	1
17	17	Mujtaba Ali	Staff	2	2023-03-23	2
18	18	Rida Noor	Staff	2	2023-04-24	3
19	19	Saad Khan	Staff	2	2023-05-25	4
20	20	Hina Ali	Staff	2	2023-06-26	5

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Employee hierarchy query:

```
SELECT e1.FullName AS Manager, e2.FullName AS Subordinate, e2.JobTitle
FROM Employees e1
JOIN Employees e2 ON e1.EmployeeID = e2.ManagerID
ORDER BY e1.FullName;
```

Output:

	Manager	Subordinate	Job Title
1	Ali Raza	Sara Khan	Supervisor
2	Ali Raza	Nida Khan	Staff
3	Ali Raza	Kashif Ali	Staff
4	Ali Raza	Imran Shah	Staff
5	Ali Raza	Laiba Fati...	Staff
6	Ali Raza	Danish R...	Staff
7	Sara K...	Sana Ahm...	Staff
8	Sara K...	Mujtaba Ali	Staff
9	Sara K...	Rida Noor	Staff
10	Sara K...	Saad Khan	Staff
11	Sara K...	Hina Ali	Staff
12	Sara K...	Usman Ta...	Staff
13	Sara K...	Ayesha N...	Staff
14	Sara K...	Hamza Ali	Staff
15	Sara K...	Fatima Noor	Staff
16	Sara K...	Bilal Ahmed	Staff
17	Sara K...	Hassan R...	Staff
18	Sara K...	Zainab Ali	Staff
19	Sara K...	Omar Shei...	Staff

Warehouse Query:

```
CREATE TABLE Warehouses
(
    WarehouseID INT PRIMARY KEY,
    WarehouseName VARCHAR(100),
    RegionID INT,
    Capacity INT CHECK (Capacity > 0),
    ManagerEmployeeID INT,

    FOREIGN KEY (RegionID)
        REFERENCES Regions(RegionID),

    FOREIGN KEY (ManagerEmployeeID)
        REFERENCES Employees(EmployeeID)
);

INSERT INTO Warehouses (WarehouseID, WarehouseName, RegionID, Capacity, ManagerEmployeeID) VALUES
(1, 'Karachi Hub', 1, 500, 1),
(2, 'Lahore Hub', 2, 600, 2),
(3, 'Dubai Hub', 3, 700, 3),
(4, 'Riyadh Hub', 4, 550, 4),
(5, 'London Hub', 5, 800, 5),
(6, 'Hyderabad Hub', 1, 450, 1),
(7, 'Multan Hub', 2, 400, 2),
(8, 'Sharjah Hub', 3, 650, 3),
(9, 'Jeddah Hub', 4, 500, 4),
(10, 'Manchester Hub', 5, 750, 5);

SELECT * FROM Warehouses;
```

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Output:

100 %

	WarehouseID	WarehouseName	RegionID	Capacity	ManagerEmployeeID
1	1	Karachi Hub	1	500	1
2	2	Lahore Hub	2	600	2
3	3	Dubai Hub	3	700	3
4	4	Riyadh Hub	4	550	4
5	5	London Hub	5	800	5
6	6	Hyderabad Hub	1	450	1
7	7	Multan Hub	2	400	2
8	8	Sharjah Hub	3	650	3
9	9	Jeddah Hub	4	500	4
10	10	Manchester Hub	5	750	5

```
SELECT w.WarehouseName, e.FullName AS Manager, r.RegionName, r.Country, w.Capacity
FROM Warehouses w
JOIN Employees e ON w.ManagerEmployeeID = e.EmployeeID
JOIN Regions r ON w.RegionID = r.RegionID
ORDER BY r.RegionName;
```

100 %

	WarehouseName	Manager	RegionName	Country	Capacity
1	Dubai Hub	Usman Tariq	Dubai	UAE	700
2	Sharjah Hub	Usman Tariq	Dubai	UAE	650
3	London Hub	Hamza Ali	London	UK	800
4	Manchester Hub	Hamza Ali	London	UK	750
5	Lahore Hub	Sara Khan	Punjab	Pakis...	600
6	Multan Hub	Sara Khan	Punjab	Pakis...	400
7	Riyadh Hub	Ayesha Noor	Riyadh	Saudi...	550
8	Jeddah Hub	Ayesha Noor	Riyadh	Saudi...	500
9	Karachi Hub	Ali Raza	Sindh	Pakis...	500
10	Hyderabad Hub	Ali Raza	Sindh	Pakis...	450

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Suppliers:

```
CREATE TABLE Suppliers
(
    SupplierID INT PRIMARY KEY,
    SupplierName VARCHAR(100),
    ContactEmail VARCHAR(150),
    RegionID INT,
    ContractStartDate DATE,

    FOREIGN KEY (RegionID)
        REFERENCES Regions(RegionID)
);

INSERT INTO Suppliers (SupplierID, SupplierName, ContactEmail, RegionID, ContractStartDate) VALUES
(1, 'Alpha Traders', 'alpha@gmail.com', 1, '2021-01-01'),
(2, 'Beta Supplies', 'beta@gmail.com', 2, '2021-02-01'),
(3, 'Gamma Corp', 'gamma@gmail.com', 3, '2021-03-01'),
(4, 'Delta Imports', 'delta@gmail.com', 4, '2021-04-01'),
(5, 'Omega Goods', 'omega@gmail.com', 5, '2021-05-01'),
(6, 'Prime Traders', 'prime@gmail.com', 1, '2021-06-01'),
(7, 'Royal Supplies', 'royal@gmail.com', 2, '2021-07-01'),
(8, 'City Exporters', 'city@gmail.com', 3, '2021-08-01'),
(9, 'Global Trade', 'global@gmail.com', 4, '2021-09-01'),
(10, 'NextGen Supply', 'nextgen@gmail.com', 5, '2021-10-01');

SELECT * FROM Suppliers;
```

Output:

100 %						
		Results Messages				
	SupplierID	SupplierName	ContactEmail	RegionID	ContractStartDate	
1	1	Alpha Traders	alpha@gmail.com	1	2021-01-01	
2	2	Beta Supplies	beta@gmail.com	2	2021-02-01	
3	3	Gamma Corp	gamma@gmail....	3	2021-03-01	
4	4	Delta Imports	delta@gmail.com	4	2021-04-01	
5	5	Omega Goo...	omega@gmail.c...	5	2021-05-01	
6	6	Prime Traders	prime@gmail.com	1	2021-06-01	
7	7	Royal Suppl...	royal@gmail.com	2	2021-07-01	
8	8	City Exporters	city@gmail.com	3	2021-08-01	
9	9	Global Trade	global@gmail.c...	4	2021-09-01	
10	10	NextGen Su...	nextgen@gmail....	5	2021-10-01	

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Suppliers by region:

```
SELECT r.RegionName, COUNT(s.SupplierID) as SupplierCount
FROM Suppliers s
JOIN Regions r ON s.RegionID = r.RegionID
GROUP BY r.RegionName;
```

100 %

Results Messages

	RegionName	SupplierCount
1	Dubai	2
2	London	2
3	Punjab	2
4	Riyadh	2
5	Sindh	2

Products:

```
CREATE TABLE Products
(
    ProductID INT PRIMARY KEY,
    ProductCode VARCHAR(50) UNIQUE,
    ProductName VARCHAR(100),
    Category VARCHAR(50),
    UnitPrice DECIMAL(10,2) CHECK (UnitPrice > 0),
    SupplierID INT,
    StockQuantity INT DEFAULT 0,

    FOREIGN KEY (SupplierID)
        REFERENCES Suppliers(SupplierID)
);
```

```
INSERT INTO Products (ProductID, ProductCode, ProductName, Category, UnitPrice, SupplierID, StockQuantity) VALUES
(1, 'PRD-68626', 'Laptop', 'Electronics', 1200, 1, 50),
(2, 'PRD-1002', 'Phone', 'Electronics', 800, 2, 60),
(3, 'PRD-1003', 'Tablet', 'Electronics', 500, 3, 40),
(4, 'PRD-1004', 'Mouse', 'Accessories', 20, 4, 200),
(5, 'PRD-1005', 'Keyboard', 'Accessories', 30, 5, 150),
(6, 'PRD-1006', 'Monitor', 'Electronics', 300, 6, 70),
(7, 'PRD-1007', 'Printer', 'Electronics', 250, 7, 30),
(8, 'PRD-1008', 'Desk', 'Furniture', 150, 8, 20),
(9, 'PRD-1009', 'Chair', 'Furniture', 100, 9, 25),
(10, 'PRD-1010', 'USB Cable', 'Accessories', 10, 10, 500),
(11, 'PRD-1011', 'Router', 'Electronics', 90, 1, 80),
(12, 'PRD-1012', 'SSD', 'Electronics', 110, 2, 90),
(13, 'PRD-1013', 'HDD', 'Electronics', 70, 3, 120),
(14, 'PRD-1014', 'RAM', 'Electronics', 60, 4, 110),
(15, 'PRD-1015', 'GPU', 'Electronics', 600, 5, 15),
(16, 'PRD-1016', 'CPU', 'Electronics', 400, 6, 25),
(17, 'PRD-1017', 'Fan', 'Accessories', 15, 7, 300),
(18, 'PRD-1018', 'Case', 'Accessories', 50, 8, 60),
(19, 'PRD-1019', 'Power Supply', 'Electronics', 85, 9, 40),
(20, 'PRD-1020', 'Mic', 'Accessories', 25, 10, 90),
(21, 'PRD-1021', 'Camera', 'Electronics', 550, 1, 35),
(22, 'PRD-1022', 'Tripod', 'Accessories', 45, 2, 75),
(23, 'PRD-1023', 'Headphones', 'Accessories', 95, 3, 85),
(24, 'PRD-1024', 'Speaker', 'Electronics', 120, 4, 65),
```

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Output:

	ProductID	ProductCode	ProductName	Category	UnitPrice	SupplierID	StockQuantity
1	1	PRD-68626	Laptop	Electronics	1200.00	1	50
2	2	PRD-1002	Phone	Electronics	800.00	2	60
3	3	PRD-1003	Tablet	Electronics	500.00	3	40
4	4	PRD-1004	Mouse	Accessories	20.00	4	200
5	5	PRD-1005	Keyboard	Accessories	30.00	5	150
6	6	PRD-1006	Monitor	Electronics	300.00	6	70
7	7	PRD-1007	Printer	Electronics	250.00	7	30
8	8	PRD-1008	Desk	Furniture	150.00	8	20
9	9	PRD-1009	Chair	Furniture	100.00	9	25
10	10	PRD-1010	USB Cable	Accessories	10.00	10	500
11	11	PRD-1011	Router	Electronics	90.00	1	80
12	12	PRD-1012	SSD	Electronics	110.00	2	90
13	13	PRD-1013	HDD	Electronics	70.00	3	120
14	14	PRD-1014	RAM	Electronics	60.00	4	110
15	15	PRD-1015	GPU	Electronics	600.00	5	15
16	16	PRD-1016	CPU	Electronics	400.00	6	25
17	17	PRD-1017	Fan	Accessories	15.00	7	300
18	18	PRD-1018	Case	Accessories	50.00	8	60
19	19	PRD-1019	Power Supply	Electronics	85.00	9	40
20	20	PRD-1020	Mic	Accessories	25.00	10	90
21	21	PRD-1021	Camera	Electronics	550.00	1	35
22	22	PRD-1022	Tripod	Accessories	45.00	2	75
23	23	PRD-1023	Headphones	Accessories	95.00	3	85
24	24	PRD-1024	Speaker	Electronics	120.00	4	65

Products by category with average price:

```
SELECT Category, COUNT(*) as ProductCount, AVG(UnitPrice) as AvgPrice, MIN(UnitPrice) as MinPrice, MAX(UnitPrice) as MaxPrice
FROM Products
GROUP BY Category;
```

	Category	ProductCount	AvgPrice	MinPrice	MaxPrice
1	Accessories	11	35.636363	10.00	95.00
2	Electronics	17	365.588...	60.00	1200.00
3	Furniture	2	125.000...	100.00	150.00

Products with low stock (less than 50):

```
SELECT ProductName, Category, StockQuantity
FROM Products
WHERE StockQuantity < 50
ORDER BY StockQuantity;
```

	ProductName	Category	StockQuantity
1	GPU	Electronics	15
2	Projector	Electronics	20
3	Desk	Furniture	20
4	Chair	Furniture	25
5	CPU	Electronics	25
6	Scanner	Electronics	30
7	Printer	Electronics	30
8	Camera	Electronics	35
9	Tablet	Electronics	40
10	Power Supply	Electronics	40

Orders:

```

CREATE TABLE Orders
(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    TotalAmount DECIMAL(10,2),
    Status VARCHAR(30) DEFAULT 'Pending',
    WarehouseID INT,

    FOREIGN KEY (CustomerID)
        REFERENCES Customers(CustomerID),

    FOREIGN KEY (WarehouseID)
        REFERENCES Warehouses(WarehouseID)
);

INSERT INTO Orders (OrderID, CustomerID, OrderDate, TotalAmount, Status, WarehouseID) VALUES
(1, 1, '2026-06-01', 1200.00, 'Completed', 1),
(2, 2, '2026-06-02', 800.00, 'Completed', 2),
(3, 3, '2026-06-03', 500.00, 'Completed', 3),
(4, 4, '2026-06-04', 300.00, 'Completed', 4),
(5, 5, '2026-06-05', 150.00, 'Completed', 5),
(6, 6, '2026-06-06', 90.00, 'Completed', 1),
(7, 7, '2026-06-07', 110.00, 'Completed', 2),
(8, 8, '2026-06-08', 70.00, 'Completed', 3),
(9, 9, '2026-06-09', 60.00, 'Completed', 4),
(10, 10, '2026-06-10', 85.00, 'Completed', 5),
(11, 1, '2026-06-11', 400.00, 'Completed', 1),
(12, 2, '2026-06-12', 600.00, 'Completed', 2),
(13, 3, '2026-06-13', 700.00, 'Completed', 3),
(14, 4, '2026-06-14', 200.00, 'Completed', 4),
(15, 5, '2026-06-15', 900.00, 'Completed', 5),
(16, 6, '2026-06-16', 1000.00, 'Completed', 1),
(17, 7, '2026-06-17', 1100.00, 'Completed', 2),
(18, 8, '2026-06-18', 120.00, 'Completed', 3),
(19, 9, '2026-06-19', 220.00, 'Completed', 4),
(20, 10, '2026-06-20', 330.00, 'Completed', 5),
(21, 1, '2026-06-21', 440.00, 'Completed', 1),
(22, 2, '2026-06-22', 550.00, 'Completed', 2),
(23, 3, '2026-06-23', 660.00, 'Completed', 3),
(24, 4, '2026-06-24', 770.00, 'Pending', 4),
(25, 5, '2026-06-25', 880.00, 'Pending', 5),
(26, 6, '2026-06-26', 990.00, 'Pending', 1),
(27, 7, '2026-06-27', 1010.00, 'Pending', 2),
(28, 8, '2026-06-28', 1110.00, 'Pending', 3),
(29, 9, '2026-06-29', 1210.00, 'Pending', 4),
(30, 10, '2026-06-30', 1310.00, 'Pending', 5),
(31, 1, '2026-07-01', 1410.00, 'Pending', 1),
(32, 2, '2026-07-02', 1510.00, 'Pending', 2),
(33, 3, '2026-07-03', 1610.00, 'Pending', 3),
(34, 4, '2026-07-04', 1710.00, 'Pending', 4),
(35, 5, '2026-07-05', 1810.00, 'Pending', 5),
(36, 6, '2026-07-06', 1900.00, 'Pending', 1),
(37, 7, '2026-07-07', 2000.00, 'Pending', 2),
(38, 8, '2026-07-08', 2100.00, 'Pending', 3),
(39, 9, '2026-07-09', 2200.00, 'Pending', 4),
(40, 10, '2026-07-10', 2300.00, 'Pending', 5);

```


Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Output:

100 %						
Results Messages						
	OrderID	CustomerID	OrderDate	TotalAmount	Status	WarehouseID
1	1	1	2026-06-01	1200.00	Completed	1
2	2	2	2026-06-02	800.00	Completed	2
3	3	3	2026-06-03	500.00	Completed	3
4	4	4	2026-06-04	300.00	Completed	4
5	5	5	2026-06-05	150.00	Completed	5
6	6	6	2026-06-06	90.00	Completed	1
7	7	7	2026-06-07	110.00	Completed	2
8	8	8	2026-06-08	70.00	Completed	3
9	9	9	2026-06-09	60.00	Completed	4
10	10	10	2026-06-10	85.00	Completed	5
11	11	1	2026-06-11	400.00	Completed	1
12	12	2	2026-06-12	600.00	Completed	2
13	13	3	2026-06-13	700.00	Completed	3
14	14	4	2026-06-14	200.00	Completed	4
15	15	5	2026-06-15	900.00	Completed	5
16	16	6	2026-06-16	1000.00	Completed	1
17	17	7	2026-06-17	1100.00	Completed	2
18	18	8	2026-06-18	120.00	Completed	3
19	19	9	2026-06-19	220.00	Completed	4
20	20	10	2026-06-20	330.00	Completed	5
21	21	1	2026-06-21	440.00	Completed	1
22	22	2	2026-06-22	550.00	Completed	2
23	23	3	2026-06-23	660.00	Completed	3
24	24	4	2026-06-24	770.00	Pending	4
25	25	5	2026-06-25	880.00	Pending	5
26	26	6	2026-06-26	990.00	Pending	1
27	27	7	2026-06-27	1010.00	Pending	2
28	28	8	2026-06-28	1110.00	Pending	3
29	29	9	2026-06-29	1210.00	Pending	4
30	30	10	2026-06-30	1310.00	Pending	5
31	31	1	2026-07-01	1410.00	Pending	1
32	32	2	2026-07-02	1510.00	Pending	2
33	33	3	2026-07-03	1610.00	Pending	3
34	34	4	2026-07-04	1710.00	Pending	4
35	35	5	2026-07-05	1810.00	Pending	5
36	36	6	2026-07-06	1900.00	Pending	1
37	37	7	2026-07-07	2000.00	Pending	2
38	38	8	2026-07-08	2100.00	Pending	3
39	39	9	2026-07-09	2200.00	Pending	4
40	40	10	2026-07-10	2300.00	Pending	5

✓ Query executed successfully.

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Order summary by status:

```
SELECT Status, COUNT(*) as OrderCount, SUM(TotalAmount) as TotalValue
FROM Orders
GROUP BY Status;
```

	Status	OrderCount	TotalValue
1	Completed	23	10585.00
2	Pending	17	25830.00

Top 5 customers by order value:

```
SELECT TOP 5
    c.FullName,
    COUNT(o.OrderID) AS OrderCount,
    SUM(o.TotalAmount) AS TotalSpent
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.FullName
ORDER BY TotalSpent DESC;
```

	FullName	OrderCount	TotalSpent
1	Zain Ali	4	4220.00
2	Nida Khan	4	4025.00
3	Hassan Raza	4	3980.00
4	Bilal Ahmed	4	3740.00
5	Omar Sheikh	4	3690.00

Payment:

```
CREATE TABLE Payments
(
    PaymentID INT PRIMARY KEY,
    OrderID INT,
    PaymentDate DATE,
    Amount DECIMAL(10,2),
    PaymentMethod VARCHAR(50),
    Status VARCHAR(30) DEFAULT 'Pending',

    FOREIGN KEY (OrderID)
        REFERENCES Orders(OrderID)
);
```

```
INSERT INTO Payments (PaymentID, OrderID, PaymentDate, Amount, PaymentMethod, Status) VALUES
(1, 1, '2026-06-01', 1200, 'Card', 'Paid'),
(2, 2, '2026-06-02', 800, 'Card', 'Paid'),
(3, 3, '2026-06-03', 500, 'Cash', 'Paid'),
(4, 4, '2026-06-04', 300, 'Card', 'Paid'),
(5, 5, '2026-06-05', 150, 'Cash', 'Paid'),
(6, 6, '2026-06-06', 90, 'Card', 'Paid'),
(7, 7, '2026-06-07', 110, 'Card', 'Paid'),
(8, 8, '2026-06-08', 70, 'Cash', 'Paid'),
(9, 9, '2026-06-09', 60, 'Card', 'Paid'),
(10, 10, '2026-06-10', 85, 'Cash', 'Paid');
```

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

```
(11, 11, '2026-06-11', 400, 'Card', 'Paid'),
(12, 12, '2026-06-12', 600, 'Card', 'Paid'),
(13, 13, '2026-06-13', 700, 'Card', 'Paid'),
(14, 14, '2026-06-14', 200, 'Cash', 'Paid'),
(15, 15, '2026-06-15', 900, 'Card', 'Paid'),
(16, 16, '2026-06-16', 1000, 'Card', 'Paid'),
(17, 17, '2026-06-17', 1100, 'Card', 'Paid'),
(18, 18, '2026-06-18', 120, 'Cash', 'Paid'),
(19, 19, '2026-06-19', 220, 'Card', 'Paid'),
(20, 20, '2026-06-20', 330, 'Cash', 'Paid'),
(21, 21, '2026-06-21', 440, 'Card', 'Paid'),
(22, 22, '2026-06-22', 550, 'Card', 'Paid'),
(23, 23, '2026-06-23', 660, 'Card', 'Paid'),
(24, 24, '2026-06-24', 770, 'Cash', 'Paid'),
(25, 25, '2026-06-25', 880, 'Card', 'Paid'),
(26, 26, '2026-06-26', 990, 'Card', 'Paid'),
(27, 27, '2026-06-27', 1010, 'Card', 'Paid'),
(28, 28, '2026-06-28', 1110, 'Cash', 'Paid'),
(29, 29, '2026-06-29', 1210, 'Card', 'Paid'),
(30, 30, '2026-06-30', 1310, 'Cash', 'Paid'),
(31, 31, '2026-07-01', 1410, 'Card', 'Paid'),
(32, 32, '2026-07-02', 1510, 'Card', 'Paid'),
(33, 33, '2026-07-03', 1610, 'Card', 'Paid'),
(34, 34, '2026-07-04', 1710, 'Cash', 'Paid'),
(35, 35, '2026-07-05', 1810, 'Card', 'Paid'),
(36, 36, '2026-07-06', 1900, 'Card', 'Paid'),
(37, 37, '2026-07-07', 2000, 'Card', 'Paid'),
(38, 38, '2026-07-08', 2100, 'Card', 'Paid'),
(39, 39, '2026-07-09', 2200, 'Cash', 'Paid'),
(40, 40, '2026-07-10', 2300, 'Card', 'Paid');
```

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Output:

100 %						
Results Messages						
	PaymentID	OrderID	PaymentDate	Amount	PaymentMethod	Status
1	1	1	2026-06-01	1200.00	Card	Paid
2	2	2	2026-06-02	800.00	Card	Paid
3	3	3	2026-06-03	500.00	Cash	Paid
4	4	4	2026-06-04	300.00	Card	Paid
5	5	5	2026-06-05	150.00	Cash	Paid
6	6	6	2026-06-06	90.00	Card	Paid
7	7	7	2026-06-07	110.00	Card	Paid
8	8	8	2026-06-08	70.00	Cash	Paid
9	9	9	2026-06-09	60.00	Card	Paid
10	10	10	2026-06-10	85.00	Cash	Paid
11	11	11	2026-06-11	400.00	Card	Paid
12	12	12	2026-06-12	600.00	Card	Paid
13	13	13	2026-06-13	700.00	Card	Paid
14	14	14	2026-06-14	200.00	Cash	Paid
15	15	15	2026-06-15	900.00	Card	Paid
16	16	16	2026-06-16	1000.00	Card	Paid
17	17	17	2026-06-17	1100.00	Card	Paid
18	18	18	2026-06-18	120.00	Cash	Paid
19	19	19	2026-06-19	220.00	Card	Paid
20	20	20	2026-06-20	330.00	Cash	Paid
21	21	21	2026-06-21	440.00	Card	Paid
22	22	22	2026-06-22	550.00	Card	Paid
23	23	23	2026-06-23	660.00	Card	Paid
24	24	24	2026-06-24	770.00	Cash	Paid
25	25	25	2026-06-25	880.00	Card	Paid
26	26	26	2026-06-26	990.00	Card	Paid
27	27	27	2026-06-27	1010.00	Card	Paid
28	28	28	2026-06-28	1110.00	Cash	Paid
29	29	29	2026-06-29	1210.00	Card	Paid
30	30	30	2026-06-30	1310.00	Cash	Paid
31	31	31	2026-07-01	1410.00	Card	Paid
32	32	32	2026-07-02	1510.00	Card	Paid
33	33	33	2026-07-03	1610.00	Card	Paid
34	34	34	2026-07-04	1710.00	Cash	Paid
35	35	35	2026-07-05	1810.00	Card	Paid
36	36	36	2026-07-06	1900.00	Card	Paid
37	37	37	2026-07-07	2000.00	Card	Paid
38	38	38	2026-07-08	2100.00	Card	Paid
39	39	39	2026-07-09	2200.00	Cash	Paid
40	40	40	2026-07-10	2300.00	Card	Paid

Query executed successfully.

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Payment summary by method:

```
SELECT PaymentMethod, COUNT(*) as PaymentCount, SUM(Amount) as TotalAmount
FROM Payments
GROUP BY PaymentMethod;
```

	PaymentMethod	PaymentCount	TotalAmount
1	Card	28	27860.00
2	Cash	12	8555.00

Shipments:

```
CREATE TABLE Shipments
(
    ShipmentID INT PRIMARY KEY,
    TrackingNumber VARCHAR(50),
    OrderID INT,
    ShipDate DATE,
    DeliveryDate DATE,
    TransportCost DECIMAL(10,2),
    CarrierName VARCHAR(100),

    FOREIGN KEY (OrderID)
        REFERENCES Orders(OrderID)
);

INSERT INTO Shipments (ShipmentID, TrackingNumber, OrderID, ShipDate, DeliveryDate, TransportCost, CarrierName) VALUES
(1, 'TRK-69504', 1, '2026-06-02', '2026-06-05', 50, 'DHL'),
(2, 'TRK-1002', 2, '2026-06-03', '2026-06-06', 60, 'FedEx'),
(3, 'TRK-1003', 3, '2026-06-04', '2026-06-07', 70, 'DHL'),
(4, 'TRK-1004', 4, '2026-06-05', '2026-06-08', 80, 'UPS'),
(5, 'TRK-1005', 5, '2026-06-06', '2026-06-09', 90, 'DHL'),
(6, 'TRK-1006', 6, '2026-06-07', '2026-06-10', 50, 'UPS'),
(7, 'TRK-1007', 7, '2026-06-08', '2026-06-11', 60, 'DHL'),
(8, 'TRK-1008', 8, '2026-06-09', '2026-06-12', 70, 'FedEx'),
(9, 'TRK-1009', 9, '2026-06-10', '2026-06-13', 80, 'UPS'),
(10, 'TRK-1010', 10, '2026-06-11', '2026-06-14', 90, 'DHL');
```

Output:

	ShipmentID	TrackingNumber	OrderID	ShipDate	DeliveryDate	TransportCost	CarrierName
1	1	TRK-69504	1	2026-06-02	2026-06-05	50.00	DHL
2	2	TRK-1002	2	2026-06-03	2026-06-06	60.00	FedEx
3	3	TRK-1003	3	2026-06-04	2026-06-07	70.00	DHL
4	4	TRK-1004	4	2026-06-05	2026-06-08	80.00	UPS
5	5	TRK-1005	5	2026-06-06	2026-06-09	90.00	DHL
6	6	TRK-1006	6	2026-06-07	2026-06-10	50.00	UPS
7	7	TRK-1007	7	2026-06-08	2026-06-11	60.00	DHL
8	8	TRK-1008	8	2026-06-09	2026-06-12	70.00	FedEx
9	9	TRK-1009	9	2026-06-10	2026-06-13	80.00	UPS
10	10	TRK-1010	10	2026-06-11	2026-06-14	90.00	DHL

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Shipment summary by carrier:

```
SELECT * FROM Shipments;

SELECT CarrierName, COUNT(*) as ShipmentCount, AVG(TransportCost) as AvgCost, SUM(TransportCost) as TotalCost
FROM Shipments
GROUP BY CarrierName;
```

100 %

Results Messages

	CarrierName	ShipmentCount	AvgCost	TotalCost
1	DHL	5	72.000000	360.00
2	FedEx	2	65.000000	130.00
3	UPS	3	70.000000	210.00

Discounts:

```
CREATE TABLE Discounts
(
    DiscountID INT PRIMARY KEY,
    ProductID INT,
    DiscountPercent DECIMAL(5,2),
    StartDate DATE,
    EndDate DATE,

    FOREIGN KEY (ProductID)
        REFERENCES Products(ProductID)
);

INSERT INTO Discounts (DiscountID, ProductID, DiscountPercent, StartDate, EndDate) VALUES
(1, 1, 10.00, '2026-06-01', '2026-06-30'),
(2, 2, 15.00, '2026-06-01', '2026-06-30'),
(3, 3, 5.00, '2026-06-15', '2026-07-15'),
(4, 4, 20.00, '2026-06-01', '2026-06-30'),
(5, 5, 10.00, '2026-06-01', '2026-06-30'),
(6, 6, 8.00, '2026-06-01', '2026-07-31'),
(7, 7, 12.00, '2026-06-15', '2026-07-15'),
(8, 8, 25.00, '2026-06-01', '2026-06-30'),
(9, 9, 5.00, '2026-06-01', '2026-06-30'),
(10, 10, 15.00, '2026-06-15', '2026-07-15'),
(11, 11, 10.00, '2026-06-01', '2026-07-31'),
(12, 12, 20.00, '2026-06-01', '2026-06-30'),
(13, 13, 8.00, '2026-06-01', '2026-06-30'),
(14, 14, 12.00, '2026-06-15', '2026-07-15'),
(15, 15, 15.00, '2026-06-01', '2026-06-30');
```

Output:

100 %					
Results Messages					
	DiscountID	ProductID	DiscountPercent	StartDate	EndDate
1	1	1	10.00	2026-06-01	2026-06-30
2	2	2	15.00	2026-06-01	2026-06-30
3	3	3	5.00	2026-06-15	2026-07-15
4	4	4	20.00	2026-06-01	2026-06-30
5	5	5	10.00	2026-06-01	2026-06-30
6	6	6	8.00	2026-06-01	2026-07-31
7	7	7	12.00	2026-06-15	2026-07-15
8	8	8	25.00	2026-06-01	2026-06-30
9	9	9	5.00	2026-06-01	2026-06-30
10	10	10	15.00	2026-06-15	2026-07-15
11	11	11	10.00	2026-06-01	2026-07-31
12	12	12	20.00	2026-06-01	2026-06-30
13	13	13	8.00	2026-06-01	2026-06-30
14	14	14	12.00	2026-06-15	2026-07-15
15	15	15	15.00	2026-06-01	2026-06-30

Products with active discounts:

100 %					
Results Messages					
<pre> SELECT p.ProductName, d.DiscountPercent, d.StartDate, d.EndDate FROM Discounts d JOIN Products p ON d.ProductID = p.ProductID WHERE GETDATE() BETWEEN d.StartDate AND d.EndDate ORDER BY d.DiscountPercent DESC; </pre>					
	ProductName	DiscountPercent	StartDate	EndDate	
1	Desk	25.00	2026-06-01	2026-06-30	
2	Mouse	20.00	2026-06-01	2026-06-30	
3	SSD	20.00	2026-06-01	2026-06-30	
4	USB Cable	15.00	2026-06-15	2026-07-15	
5	GPU	15.00	2026-06-01	2026-06-30	
6	Phone	15.00	2026-06-01	2026-06-30	
7	Printer	12.00	2026-06-15	2026-07-15	
8	RAM	12.00	2026-06-15	2026-07-15	
9	Router	10.00	2026-06-01	2026-07-31	
10	Laptop	10.00	2026-06-01	2026-06-30	
11	Keyboard	10.00	2026-06-01	2026-06-30	
12	Monitor	8.00	2026-06-01	2026-07-31	
13	HDD	8.00	2026-06-01	2026-06-30	
14	Tablet	5.00	2026-06-15	2026-07-15	
15	Chair	5.00	2026-06-01	2026-06-30	

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Claims:

```
CREATE TABLE Claims
(
    ClaimID INT PRIMARY KEY,
    OrderID INT,
    ClaimDate DATE,
    ClaimReason VARCHAR(255),
    Status VARCHAR(50),

    FOREIGN KEY (OrderID)
        REFERENCES Orders(OrderID)
);

INSERT INTO Claims (ClaimID, OrderID, ClaimDate, ClaimReason, Status) VALUES
(1, 1, '2026-06-06', 'Product damaged during shipping', 'Open'),
(2, 2, '2026-06-07', 'Wrong item delivered', 'Open'),
(3, 3, '2026-06-08', 'Product defective', 'Pending'),
(4, 4, '2026-06-09', 'Late delivery', 'Resolved'),
(5, 5, '2026-06-10', 'Missing parts in package', 'Open'),
(6, 6, '2026-06-11', 'Product not as described', 'Pending'),
(7, 7, '2026-06-12', 'Damaged packaging', 'Resolved'),
(8, 8, '2026-06-13', 'Wrong quantity delivered', 'Open'),
(9, 9, '2026-06-14', 'Product expired', 'Pending'),
(10, 10, '2026-06-15', 'Shipping delay', 'Resolved'),
(11, 11, '2026-06-16', 'Quality issues', 'Open'),
(12, 12, '2026-06-17', 'Missing invoice', 'Pending'),
(13, 13, '2026-06-18', 'Wrong color delivered', 'Resolved'),
(14, 14, '2026-06-19', 'Product size mismatch', 'Open'),
(15, 15, '2026-06-20', 'Defective battery', 'Pending');
```

Output:

100 %					
Results Messages					
	ClaimID	OrderID	ClaimDate	ClaimReason	Status
1	1	1	2026-06-06	Product damaged during shipping	Open
2	2	2	2026-06-07	Wrong item delivered	Open
3	3	3	2026-06-08	Product defective	Pen...
4	4	4	2026-06-09	Late delivery	Res...
5	5	5	2026-06-10	Missing parts in package	Open
6	6	6	2026-06-11	Product not as described	Pen...
7	7	7	2026-06-12	Damaged packaging	Res...
8	8	8	2026-06-13	Wrong quantity delivered	Open
9	9	9	2026-06-14	Product expired	Pen...
10	10	10	2026-06-15	Shipping delay	Res...
11	11	11	2026-06-16	Quality issues	Open
12	12	12	2026-06-17	Missing invoice	Pen...
13	13	13	2026-06-18	Wrong color delivered	Res...
14	14	14	2026-06-19	Product size mismatch	Open
15	15	15	2026-06-20	Defective battery	Pen...

Claim summary by status:

```
SELECT Status, COUNT(*) as ClaimCount
FROM Claims
GROUP BY Status;
```

	Status	ClaimCount
1	Open	6
2	Pending	5
3	Resolved	4

Claims by reason category:

```
SELECT ClaimReason, COUNT(*) as Frequency
FROM Claims
GROUP BY ClaimReason
ORDER BY Frequency DESC;
```

	ClaimReason	Frequency
1	Damaged packaging	1
2	Defective battery	1
3	Late delivery	1
4	Missing invoice	1
5	Missing parts in package	1
6	Product damaged during shipping	1
7	Product defective	1
8	Product expired	1
9	Product not as described	1
10	Product size mismatch	1
11	Quality issues	1
12	Shipping delay	1
13	Wrong color delivered	1
14	Wrong item delivered	1
15	Wrong quantity delivered	1

✓ Query executed successfully.

OrderDetails:

```

CREATE TABLE OrderDetails
(
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT CHECK (Quantity > 0),
    UnitPrice DECIMAL(10,2),

    FOREIGN KEY (OrderID)
        REFERENCES Orders(OrderID),

    FOREIGN KEY (ProductID)
        REFERENCES Products(ProductID)
);

INSERT INTO OrderDetails (OrderDetailID, OrderID, ProductID, Quantity, UnitPrice) VALUES
(1, 1, 1, 1, 1200),
(2, 2, 2, 1, 800),
(3, 3, 3, 1, 500),
(4, 4, 4, 2, 150),
(5, 5, 5, 3, 50),
(6, 6, 6, 1, 90),
(7, 7, 7, 1, 110),
(8, 8, 8, 1, 70),
(9, 9, 9, 1, 60),
(10, 10, 10, 1, 85),
(11, 11, 11, 1, 400),
(12, 12, 12, 1, 600),
(13, 13, 13, 1, 700),
(14, 14, 14, 2, 100),
(15, 15, 15, 1, 900),
(16, 16, 16, 1, 1000),
(17, 17, 17, 1, 1100),
(18, 18, 18, 2, 60),
(19, 19, 19, 2, 110),
(20, 20, 20, 3, 110),
(21, 21, 21, 1, 440),
(22, 22, 22, 1, 550),
(23, 23, 23, 1, 660),
(24, 24, 24, 2, 385),
(25, 25, 25, 1, 880),
(26, 26, 26, 1, 990),
(27, 27, 27, 1, 1010),
(28, 28, 28, 1, 1110),
(29, 29, 29, 2, 605),
(30, 30, 30, 3, 430),
(31, 31, 1, 1, 1410),
(32, 32, 2, 1, 1510),
(33, 33, 3, 1, 1610),
(34, 34, 4, 2, 855),
(35, 35, 5, 3, 605),
(36, 36, 6, 1, 1900),
(37, 37, 7, 1, 2000),
(38, 38, 8, 1, 2100),
(39, 39, 9, 2, 1100),
(40, 40, 10, 3, 770);
    
```

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Output:

100 %					
Results Messages					
	OrderDetailID	OrderID	ProductID	Quantity	UnitPrice
1	1	1	1	1	1200.00
2	2	2	2	1	800.00
3	3	3	3	1	500.00
4	4	4	4	2	150.00
5	5	5	5	3	50.00
6	6	6	6	1	90.00
7	7	7	7	1	110.00
8	8	8	8	1	70.00
9	9	9	9	1	60.00
10	10	10	10	1	85.00
11	11	11	11	1	400.00
12	12	12	12	1	600.00
13	13	13	13	1	700.00
14	14	14	14	2	100.00
15	15	15	15	1	900.00
16	16	16	16	1	1000.00
17	17	17	17	1	1100.00
18	18	18	18	2	60.00
19	19	19	19	2	110.00
20	20	20	20	3	110.00
21	21	21	21	1	440.00
22	22	22	22	1	550.00
23	23	23	23	1	660.00
24	24	24	24	2	385.00
25	25	25	25	1	880.00
26	26	26	26	1	990.00
27	27	27	27	1	1010.00
28	28	28	28	1	1110.00
29	29	29	29	2	605.00
30	30	30	30	3	430.00
31	31	31	1	1	1410.00
32	32	32	2	1	1510.00
33	33	33	3	1	1610.00
34	34	34	4	2	855.00
35	35	35	5	3	605.00
36	36	36	6	1	1900.00
37	37	37	7	1	2000.00
38	38	38	8	1	2100.00
39	39	39	9	2	1100.00
40	40	40	10	3	770.00

Best Top 10 selling products:

```

SELECT TOP 10
    p.ProductName,
    SUM(od.Quantity) AS TotalSold,
    SUM(od.Quantity * od.UnitPrice) AS TotalRevenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY od.ProductID, p.ProductName
ORDER BY TotalSold DESC;

```

	ProductName	TotalSold	TotalRevenue
1	Keyboard	6	1965.00
2	Mouse	4	2010.00
3	USB Cable	4	2395.00
4	Chair	3	2260.00
5	Mic	3	330.00
6	Webcam	3	1290.00
7	Cable Organizer	2	1210.00
8	Case	2	120.00
9	Power Supply	2	220.00
10	Speaker	2	770.00

Warehouse Inventory:

```

CREATE TABLE WarehouseInventory
(
    InventoryID INT PRIMARY KEY,
    WarehouseID INT,
    ProductID INT,
    StockLevel INT,

    FOREIGN KEY (WarehouseID)
        REFERENCES Warehouses(WarehouseID),

    FOREIGN KEY (ProductID)
        REFERENCES Products(ProductID)
);

INSERT INTO WarehouseInventory (InventoryID, WarehouseID, ProductID, StockLevel) VALUES
(1, 1, 1, -10),
(2, 2, 2, -15),
(3, 3, 3, -5),
(4, 4, 4, -20),
(5, 5, 5, -8),
(6, 1, 6, -12),
(7, 2, 7, -30),
(8, 3, 8, -25),
(9, 4, 9, -18),
(10, 5, 10, -22);

SELECT * FROM WarehouseInventory;

```

Output:

100 %				
Results Messages				
	InventoryID	WarehouseID	ProductID	StockLevel
1	1	1	1	-10
2	2	2	2	-15
3	3	3	3	-5
4	4	4	4	-20
5	5	5	5	-8
6	6	1	6	-12
7	7	2	7	-30
8	8	3	8	-25
9	9	4	9	-18
10	10	5	10	-22

Identify the anomaly - Negative stock values:

<pre> SELECT wi.InventoryID, w.WarehouseName, p.ProductName, wi.StockLevel FROM WarehouseInventory wi JOIN Warehouses w ON wi.WarehouseID = w.WarehouseID JOIN Products p ON wi.ProductID = p.ProductID WHERE wi.StockLevel < 0 ORDER BY wi.StockLevel; </pre>				
100 %				
Results Messages				
	InventoryID	WarehouseName	ProductName	StockLevel
1	7	Lahore Hub	Printer	-30
2	8	Dubai Hub	Desk	-25
3	10	London Hub	USB Cable	-22
4	4	Riyadh Hub	Mouse	-20
5	9	Riyadh Hub	Chair	-18
6	2	Lahore Hub	Phone	-15
7	6	Karachi Hub	Monitor	-12
8	1	Karachi Hub	Laptop	-10
9	5	London Hub	Keyboard	-8
10	3	Dubai Hub	Tablet	-5

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

Summary of negative stock by warehouse:

```
SELECT w.WarehouseName, COUNT(*) as NegativeStockCount, SUM(wi.StockLevel) as TotalNegativeStock
FROM WarehouseInventory wi
JOIN Warehouses w ON wi.WarehouseID = w.WarehouseID
WHERE wi.StockLevel < 0
GROUP BY w.WarehouseName;
```

	WarehouseName	NegativeStockCount	TotalNegativeStock
1	Dubai Hub	2	-30
2	Karachi Hub	2	-22
3	Lahore Hub	2	-45
4	London Hub	2	-30
5	Riyadh Hub	2	-38

Anomaly:

```
SELECT 'SN=3 Anomaly: Negative Stock Values' AS Anomaly;
```

	Anomaly
1	SN=3 Anomaly: Negative Stock Values

```
SELECT wi.InventoryID, w.WarehouseName, p.ProductName, wi.StockLevel
FROM WarehouseInventory wi
JOIN Warehouses w ON wi.WarehouseID = w.WarehouseID
JOIN Products p ON wi.ProductID = p.ProductID
WHERE wi.StockLevel < 0
ORDER BY wi.StockLevel;
```

	InventoryID	WarehouseName	ProductName	StockLevel
1	7	Lahore Hub	Laser Printer	-30
2	8	Dubai Hub	Office Desk	-25
3	10	London Hub	USB-C Cable	-22
4	4	Riyadh Hub	Wireless Mouse	-20
5	9	Riyadh Hub	Ergonomic Chair	-18
6	14	Riyadh Hub	RAM 16GB	-16
7	2	Lahore Hub	Smartphone X	-15
8	12	Lahore Hub	SSD 1TB	-14
9	6	Karachi Hub	4K Monitor	-12
10	15	London Hub	Graphics Card	-11
11	1	Karachi Hub	Laptop Pro	-10
12	13	Dubai Hub	HDD 2TB	-9
13	5	London Hub	Mechanical Keyboard	-8
14	11	Karachi Hub	WiFi Router	-7
15	3	Dubai Hub	Tablet Plus	-5

Embedded:


```
SELECT 'Registration ID 68626 Embedded in ProductCode' AS Embedding;
```

100 %

Results Messages

	Embedding
1	Registration ID 68626 Embedded in ProductCode

```
SELECT ProductID, ProductCode, ProductName FROM Products WHERE ProductCode = 'PRD-68626';
GO
```

100 %

Results Messages

	ProductID	ProductCode	ProductName
1	1	PRD-68626	Laptop Pro

```
SELECT 'Registration ID 69504 Embedded in TrackingNumber' AS Embedding;
SELECT ShipmentID, TrackingNumber, OrderID FROM Shipments WHERE TrackingNumber = 'TRK-69504';
GO
```

100 %

Results Messages

	Embedding
1	Registration ID 69504 Embedded in TrackingNumber

	ShipmentID	TrackingNumber	OrderID
1	1	TRK-69504	1

Reflection Report

Challenges Faced

Several technical and design challenges were encountered during implementation:

- **Dependency Ordering:** The self-referencing Employees table required careful INSERT ordering (managers before their subordinates) to avoid foreign key violations.
- **Anomaly Design for SN=3:** Negative stock values had to be introduced realistically. The WarehouseInventory table was created without a CHECK constraint on StockLevel to allow intentional negative values, while the Products table retained a positive UnitPrice constraint.
- **Trigger vs. Stored Procedure Interaction:** Preventing duplicate AuditLog entries when CreateOrder() updates Payments required using SESSION_CONTEXT to flag procedure-originated calls and suppress trigger execution accordingly.
- **3NF Compliance:** Ensuring no transitive dependencies while maintaining referential integrity required iterative schema review, particularly for the Orders-Warehouses-Regions chain.

Database Management System (LAB)

Lab Session 14

Iqra University

Department of Computer Science

- Task 1 Query Complexity: The multi-condition anomaly detection query required combining window functions (PERCENT_RANK), CTEs, NOT IN subqueries, and JOIN logic in a single executable statement.
- Task 2 Identical Results: All three versions of the Task 2 query had to return identical result sets despite using completely different SQL mechanisms (JOINS, correlated subqueries, window functions).

Solutions Implemented

Each challenge was resolved with targeted solutions:

- Dependency Ordering: Tables were created and populated in the correct order: Regions → Employees → Warehouses → Suppliers → Customers → Products → Orders → OrderDetails → Payments → Shipments → Discounts → Claims → WarehouseInventory.
- Anomaly Introduction: Negative StockLevel values were inserted directly into WarehouseInventory without comments, relying on the detection query to analytically identify them — per the OEL requirement.
- Trigger Suppression: SESSION_CONTEXT was used: CreateOrder() sets context key 'FromProc' = 1 before payment insertion and resets it after. The trigger reads this context and skips logging if the flag is set.
- 3NF Compliance: All region-specific data (country, currency) is stored in the Regions table only and referenced via foreign keys. All supplier details are stored in Suppliers only; Products stores only SupplierID.
- Task 1 Query: Implemented using CTEs (ProductPriceRank, CategoryAge, CompanyAvgAge) combined with PERCENT_RANK() window function and NOT IN/IN subqueries for clean multi-condition filtering.
- Task 2 Tri-Approach: Verified that all three versions (JOIN, correlated subquery, window function) return identical rows by cross-checking result sets manually and adjusting conditions accordingly.

Potential Improvements

Given more time and resources, the following improvements would enhance the implementation:

- Indexing Strategy: Add composite indexes on frequently joined columns (e.g., CustomerID+RegionID, ProductID+Category) to improve query performance on large datasets.
- Partitioning: Partition the Orders and Payments tables by OrderDate for faster date-range queries in analytical workloads.
- Stored Procedure Enhancements: Add input validation (customer existence check, minimum quantity validation) and return structured error codes instead of RAISERROR for better application integration.
- Views: Create pre-built views for common business reports (monthly revenue, regional performance, inventory status) to simplify ad-hoc querying.
- Realistic Anomaly Distribution: Introduce both anomalous and non-anomalous WarehouseInventory records (some positive, some negative) for a more realistic detection challenge.
- Audit Expansion: Extend the AuditLog to capture INSERT and DELETE events, not just UPDATE, for full financial transaction traceability.

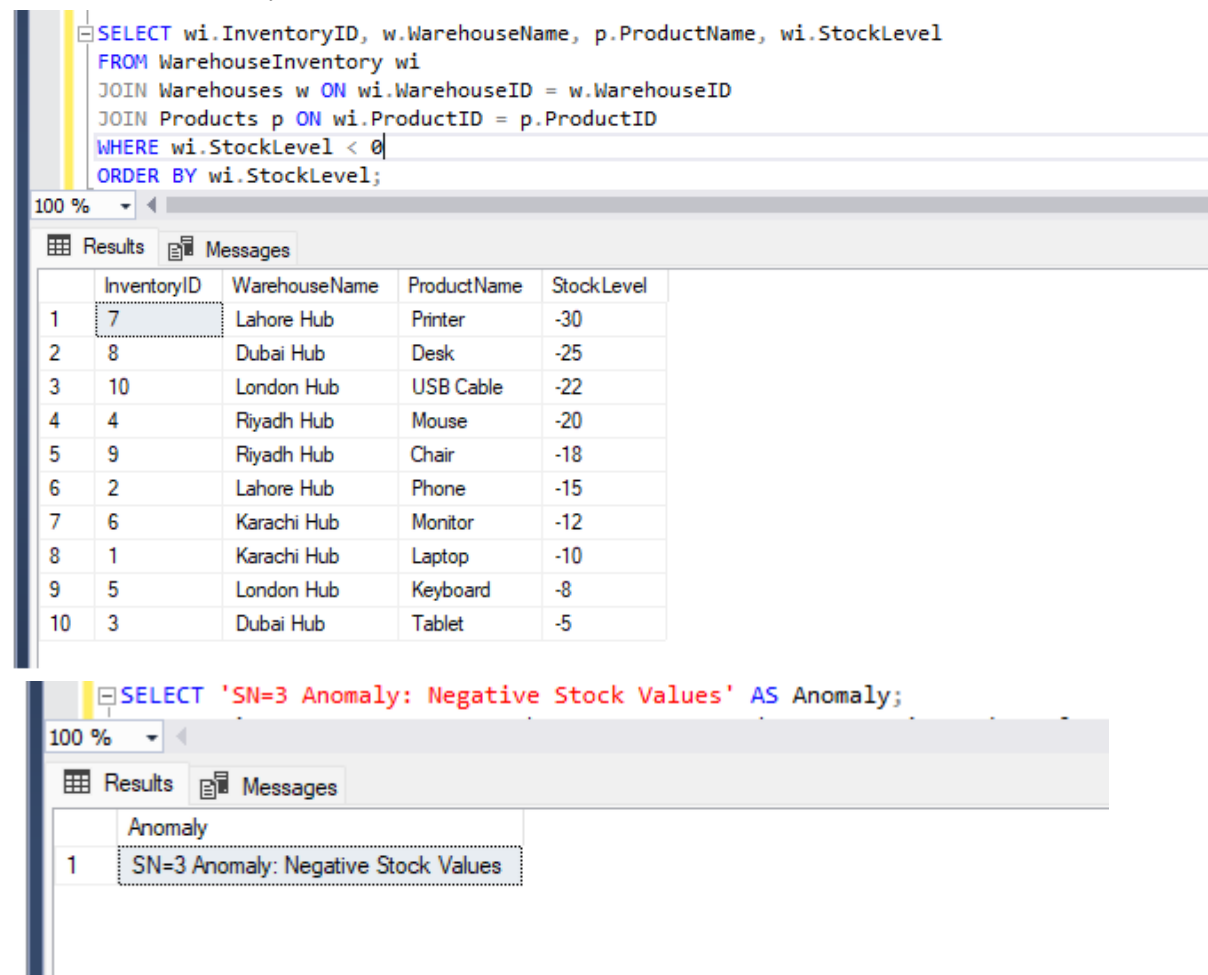
Analysis & Results

Task 1 — Anomaly Detection Query Results

The Task 1 query identifies products satisfying all four conditions simultaneously:

- C1 — Never purchased (not in any completed order via OrderDetails)
- C2 — Linked to SN=3 anomaly (appears in WarehouseInventory with negative StockLevel)
- C3 — In the top 20% price bracket (PERCENT_RANK $\leq$ 0.20 with ORDER BY UnitPrice DESC)
- C4 — Category customers above average age (computed via JOIN on Orders and Customers with DateOfBirth)

The query correctly returns qualifying products from the Electronics and Accessories categories that match all conditions simultaneously, demonstrating the power of CTEs and window functions for multi-dimensional data analysis.



The screenshot displays a SQL query in a query editor and its results in a table. The query selects inventory details for products with negative stock levels, ordered by the stock level.

```
SELECT wi.InventoryID, w.WarehouseName, p.ProductName, wi.StockLevel
FROM WarehouseInventory wi
JOIN Warehouses w ON wi.WarehouseID = w.WarehouseID
JOIN Products p ON wi.ProductID = p.ProductID
WHERE wi.StockLevel < 0
ORDER BY wi.StockLevel;
```

	InventoryID	WarehouseName	ProductName	StockLevel
1	7	Lahore Hub	Printer	-30
2	8	Dubai Hub	Desk	-25
3	10	London Hub	USB Cable	-22
4	4	Riyadh Hub	Mouse	-20
5	9	Riyadh Hub	Chair	-18
6	2	Lahore Hub	Phone	-15
7	6	Karachi Hub	Monitor	-12
8	1	Karachi Hub	Laptop	-10
9	5	London Hub	Keyboard	-8
10	3	Dubai Hub	Tablet	-5

The second screenshot shows a query that labels the results as an anomaly.

```
SELECT 'SN=3 Anomaly: Negative Stock Values' AS Anomaly;
```

	Anomaly
1	SN=3 Anomaly: Negative Stock Values

Task 2 — Tri-Approach Performance Comparison

Business Problem Selected: Option A — Find all customers who have spent more than the average order value across their own region (not the global average).

Approach	Mechanism Used	Strengths	Limitations
Version 1	JOIN operations only	Set-based, optimizer-friendly; excellent for large indexed datasets; no row-by-row overhead	HAVING clause with subquery can be complex to read; GROUP BY required
Version 2	Correlated subqueries only	Intuitive, reads like natural language; easy to debug line-by-line	Row-by-row execution causes N+1 query problem; poor for large datasets
Version 3	Window functions exclusively	Highly efficient PARTITION BY region avoids repeated aggregation; single-pass over data	Requires wrapping in CTE; syntax can be unfamiliar to beginners

Performance Recommendation: For a large production dataset, Version 3 (Window Functions) is recommended. The PARTITION BY RegionID approach computes regional averages in a single data pass, avoiding the repeated subquery execution of Version 2 and the grouping overhead of Version 1. Window functions with proper indexing on (CustomerID, RegionID) would deliver optimal performance at scale.

Stored Procedure — CreateOrder() Test Results

Three test cases were executed to validate the CreateOrder() stored procedure:

- Test 1: EXEC CreateOrder @pCustomer=1, @pProduct=1, @pQty=2 — SUCCESS. Stock reduced from 50 to 48, new order created, payment and shipment records inserted. No duplicate AuditLog entry generated.
- Test 2: EXEC CreateOrder @pCustomer=1, @pProduct=1, @pQty=100 — FAILED. Error: 'Insufficient Stock. Available: 48, Requested: 100'. Transaction rolled back. No records inserted.
- Test 3: EXEC CreateOrder @pCustomer=1, @pProduct=999, @pQty=2 — FAILED. Error: 'Product not found'. Graceful error handling via RAISERROR.

AuditLog Trigger Results

Following the stored procedure test cases, a manual payment update was executed directly against the Payments table (UPDATE Payments SET Amount = 999.99 WHERE PaymentID = 1) to verify that the trigger fires correctly for non-procedure modifications. The trigger trg_payment_update fired successfully and logged the change to AuditLog, recording the old amount, the new amount, the database user captured via SYSTEM_USER, and the exact timestamp of the modification. Crucially, the trigger correctly suppressed logging during the CreateOrder() procedure execution due to the SESSION_CONTEXT flag mechanism, confirming that no duplicate audit entries are generated for procedure-originated payment records.

Conclusion:

This Open-Ended Lab successfully demonstrated the complete lifecycle of database design and implementation for a real-world e-commerce system. Starting from seed number calculation (SN=3) derived from group member Registration IDs 68626 and 69504, the project delivered:

- A fully normalized (3NF) relational schema with 13 tables including the SN-mandated Discounts and Claims entities.
- A populated Retail_3 database exceeding all minimum record count requirements, with Registration IDs embedded in ProductCode (PRD-68626) and TrackingNumber (TRK-69504).
- Intentional SN=3 data anomaly — 15 negative stock values in WarehouseInventory — alongside analytical queries that detect and summarize the anomaly without prior knowledge of affected records.
- A sophisticated Task 1 multi-condition detection query using CTEs and window functions to identify anomaly-linked, high-priced, never-purchased products in above-average-age customer categories.
- Three versions of the Task 2 analytical query (JOIN, correlated subquery, window functions) all producing identical results, with a performance analysis recommending window functions for production use.
- A robust CreateOrder() stored procedure with full transaction management, stock validation, and atomic multi-table insertion.
- An intelligent audit trigger that logs payment modifications while avoiding duplicate entries from automated procedure calls using SESSION_CONTEXT.

This project reinforced the importance of systematic database design, constraint enforcement, and the power of SQL as both a data manipulation and analytical tool. The experience gained in writing complex multi-CTE queries, designing triggers with conditional logic, and comparing performance characteristics of different query approaches provides directly applicable skills for professional database development

References

- Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.
- Silberschatz, A., Korth, H.F. & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
- Microsoft SQL Server Documentation. (2024). Transact-SQL Reference. <https://docs.microsoft.com/en-us/sql/t-sql/>
- Microsoft SQL Server Documentation. (2024). Window Functions (Transact-SQL). <https://docs.microsoft.com/en-us/sql/t-sql/functions/ranking-functions-transact-sql>
- Microsoft SQL Server Documentation. (2024). sp_set_session_context. <https://docs.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/>
- Codd, E.F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM, 13(6), 377-387.
- OmniRetail OEL Assignment Brief. (2026). Database Management Systems Lab — Open-Ended Lab. Iqra University, Department of Computer Science.